机器人模仿学习：从 BC、DAgger 到 VLA 的实时执行

监督数据、生成式动作块、多模态策略与实时执行

2026-10-30

目录

1. 模仿学习基础：BC 与 DAgger 2
2. 动作块与生成式模仿学习 2
3. 从模仿学习到视觉–语言–动作模型（VLA） 3
4. 实时动作执行：从 RTC 到未来状态补偿 3
参考文献与延伸阅读 5

课程首页 ·English ·PDF

项目	说明
主题	模仿学习
修订日期	2026-10-30
前置知识	深度学习基础与入门机器人控制
许可	CC BY-NC-SA 4.0 ·Xiong Zeng and contributors

1. 模仿学习基础：BC 与 DAgger

1.1 一个统一的学习对象

机器人具有不可完全观测的物理状态 s_t，相机观测 o_t，本体反馈 q_t，任务条件 g。策略以历史 $h_t = (o_{\le t}, q_{\le t}, g)$ 预测动作：$a_t \sim \pi_\theta(\cdot \mid h_t)$，而真正发送给电机的命令 $u_t = \text{Controller}(a_t, q_t)$ 还受限幅、滤波与底层控制周期影响。

先定义动作语义，再谈网络：输出是关节位置、位置增量、速度、末端位姿，还是手指关节目标？坐标系、单位、时间戳和频率不匹配会使训练目标与执行命令脱节。训练集归一化统计量不能用测试集重新估算。

1.2 Behavior Cloning：监督拟合及其边界

设专家数据 $\mathcal{D}_E = \{(h_i, a_i^E)\}$，确定性 BC 训练 $\min_\theta \mathbb{E}_{\mathcal{D}}[\|f_\theta(h) - a^E\|_W^2]$。当专家动作条件分布接近单峰时十分直接；多峰示教中，均值可能落在两条有效路径之间，因此可以用潜变量或生成式策略建模完整分布。

BC 还有**分布偏移**：训练状态来自专家占据分布，部署时错误使策略进入未见状态。经典有限时域错误分析说明累计代价最坏可出现 $\mathcal{O}(T^2\varepsilon)$ 的增长；这一尺度并不能直接由连续动作 MSE 换算为真实任务成功率。

1.3 DAgger：在策略实际访问的状态上得到专家标签

DAgger 是**数据收集与训练协议**，并不是新的策略网络。轮次 k：执行当前策略（或专家混合策略）⟶ 访问新状态 ⟶ 为这些状态提供专家标签 ⟶ 聚合数据 $\mathcal{D}_{k+1} = \mathcal{D}_k \cup \Delta\mathcal{D}_k$ ⟶ 重新训练。这里的 “混合” 是选择由谁控制，而不是把专家动作与机器人动作线性混合。

在机器人上持续为所有状态标注动作可能昂贵。需要区分 “机器人真正执行的动作” 与 “专家希望执行的动作”；不可恢复的危险状态应安全停机、复位，而不是强求专家标签。

1.4 人类干预式纠错：HG-DAgger、IWR 与 RaC

方法	核心机制	最适合回答的问题
DAgger	收集策略访问状态的专家标签	如何缓解分布偏移?
HG-DAgger	人类仅在必要时接管，采集高价值纠正轨迹	如何降低交互式标注负担?
IWR	提高干预数据在监督更新中的权重	少量干预如何在大量常规数据中不被淹没?
RaC	明确区分 Recovery（恢复）和 Correction（纠正）	失败后如何先回到可继续执行的状态?

重要区别：纠错示教改善访问分布覆盖，生成式动作头改善条件动作分布的表达能力，两者解决不同层次的问题。对于接触操作，应同时记录干预时刻、接触模式、执行命令与专家标签。相关工作见文末参考文献。

2. 动作块与生成式模仿学习

2.1 Action Chunking：预测长度并非执行长度

在时刻 t，策略预测长度为 H 的动作块 $A_t = (a_{t|t}, \dots, a_{t+H-1|t})$。每次实际执行 $K \le H$ 步，然后重新观测、重新规划。H 决定预测范围，K 决定闭环反馈间隔，二者不应混淆。越长的开环执行通常越难及时响应接触变化。

2.2 ACT：Transformer + 条件 VAE

ACT 用多视角视觉和本体感知条件化 Transformer,训练时根据专家动作块编码潜变量 z,解码器预测整段动作：$\hat{A}_t = f_\theta(h_t, z)$。典型目标为 $\mathcal{L}_{\text{ACT}} = \mathcal{L}_{\text{masked-}L_1} + \beta\,\text{KL}(q_\phi(z \mid q_t, A_t^E) \,\|\, \mathcal{N}(0, I))$。末尾填充动作必须通过 mask 排除。部署时无法取得专家的未来动作块，原始 ACT 通常使用先验均值 $z = 0$。

Temporal Ensembling 将不同预测时刻对同一物理时刻 t 的动作预测按权重融合；它不是简单的相邻时间步平滑，且可能削弱接触模式切换时所需的快速反应。

2.3 Diffusion Policy 与 Flow Matching

Diffusion Policy 在噪声化动作块上学习去噪器：$\mathcal{L}_{\text{diff}} = \mathbb{E}\|\varepsilon - \varepsilon_\theta(A_{\text{noisy}}, n, h)\|^2$，推理时迭代去噪。它适合多模态动作分布，但采样计算需要纳入时延预算。

Flow Matching 构造噪声样本 A_0 与示教动作 A_1 之间的路径 $A_\tau = (1-\tau)A_0 + \tau A_1$，学习向量场：$\mathcal{L}_{\text{FM}} = \mathbb{E}\|v_\theta(A_\tau, \tau, h) - (A_1 - A_0)\|^2$。推理时数值积分 $\mathrm{d}A/\mathrm{d}\tau = v_\theta(A, \tau, h)$。生成式建模能表达多种合法动作，但**本身不保证**动作块在真实物理时间连续执行。

2.4 RDP：动作块之外还需要高频接触反馈

Reactive Diffusion Policy (RDP) 将低频的 slow latent diffusion 动作块生成器与高频的 fast tactile-feedback 解码器结合。它回答的问题不是 “如何更快生成一个新块”，而是**当前块执行期间能否立即根据触觉/力反馈修正动作**。对于灵巧手旋拧、插接和滑移抑制，可把它视作分层闭环设计的范例，而非直接将所有生成式策略替换为 RDP。

3. 从模仿学习到视觉–语言–动作模型（VLA）

3.1 VLA 扩展了条件，并不取代 BC

VLA 接受图像、本体状态和语言指令，形成表示 $z_t = F(I_{\le t}, q_{\le t}, \text{language})$，再预测动作或动作块。机器人动作监督预训练和下游监督微调通常仍属于广义 BC；DAgger 则规定何时补充策略访问状态下的专家标签。

3.2 两类常见动作头

动作接口	训练方式	优点与权衡
离散动作 tokens	自回归交叉熵；如 OpenVLA 路线	与语言 token 接口兼容，但量化与 token 数量影响精度、速率
连续动作块	Diffusion / Flow Matching；如 π0 路线	适合高维连续控制，但推理计算与异步执行须单独设计

FAST / FAST+（扩展阅读）：利用频域压缩构造动作 tokens，帮助理解为什么动作 tokenization 不等于逐关节离散化。它属于**动作表示**，不是与 RTC 竞争的实时执行机制。

3.3 数据与部署接口必须保持一致

样本应包含 (images, proprioception, instruction, embodiment-ID, expert-action-chunk, mask, timestamps)。不同机器人关节含义、动作单位和控制频率不一致，不能仅依靠零填充完成跨本体统一。若动作监督改为 VLA 后训练，预训练表征、动作头可训练参数及目标机器人校准范围要明确区分。

层次总结： BC / DAgger 决定监督来源；ACT / Diffusion / Flow 决定动作分布建模；VLA 决定多模态条件与模型架构；RTC / VLASH 等处理动作在物理时间上的执行。

4. 实时动作执行：从 RTC 到未来状态补偿

4.1 两种时间，三个长度

机器人按控制周期 Δt_c 执行动作，而策略需要端到端延迟 T_{lat}（采图、编码、排队、推理、传输）。用 $d = \left\lceil \frac{T_{\text{lat}}}{\Delta t_c} \right\rceil$ 表示推理期间经过的控制步数。须区分**预测长度 H、每次执行长度 K、推理占用步数 d**。例如周期 20 ms、端到端延迟 110 ms，则 $d = 6$。

同步执行可能等待新块；朴素异步执行可让旧块继续运行，但新旧块切换可能跳变。异步系统有两个不同失配：**动作前缀衔接**与**观测状态滞后**。

4.2 RTC 与 Training-Time RTC：先处理动作前缀

Inference-Time RTC 使新动作块生成以已经承诺执行的旧动作前缀为条件，并利用 inpainting / guidance 提升块间一致性；Guidance 可能带来额外推理开销。Training-Time RTC 在训练时纳入前缀条件，减少部署时额外 guidance 的需要，代价是训练过程和延迟范围适配需要重新设计。RTC 主要解决 “切换是否连续”，并不自动提供执行时刻的真实观测。

4.3 VLASH：执行时状态与预测时状态对齐

当新块开始执行时，机器人已按照旧块继续运动。若模型仅使用推理启动时的 q_t，动作可能基于过时本体状态。VLASH 使用旧动作块与状态动力学前推：

$$\hat{q}(t+\Delta) = \mathrm{RollForward}(q_t, A_{\mathrm{old}}, \Delta)$$

$$A_{\mathrm{new}} = \pi_\theta(o_t, \hat{q}(t+\Delta))$$

这里的 o_t 仍是可获得的**旧视觉观测**。因此，VLASH 主要补偿**本体状态的时间错位**，而不能声称完全解决视觉滞后。

关键细节：VLASH 不只是 “无需训练的状态前推”。原论文还提出 **Temporal-Offset Fine-Tuning**：训练时随机取偏移 δ，使用 $(o_t, q_{t+\delta})$ 预测执行时刻动作 $A^E_{t+\delta:t+\delta+H-1}$，使模型真正学会利用未来本体输入。未来状态前推是一项推理机制；偏移微调是强化这种条件使用的训练机制，两者需要区别讨论。

4.4 FutureRTC：进一步考虑未来视觉表征

FutureRTC 除了补偿本体状态，还用动作感知的特征传输与重建预测执行时刻视觉表示，并通过状态修正与策略一致性目标帮助预测条件与执行条件对齐。它是对 “只补偿本体状态、仍用过时图像” 的进一步处理，但预测的未来视觉表征并非真实未来相机反馈；突发外力、遮挡和新物体的不可预测性仍是局限。

4.5 RDP 与 RTR：不是所有实时问题都一样

RDP 针对**动作块内部**的高频触觉闭环。RTR（扩展阅读）针对高频动作块表示及块间平滑，采用 Reuse-then-Refine 思路。它们与 RTC / VLASH 有交集，但各自针对不同瓶颈，不能简单根据某一论文成功率进行通用排序。

方法	核心瓶颈	主要机制	注意事项
Temporal Ensembling	重叠预测不一致	同一执行时刻的预测融合	过平滑可能掩盖突然接触
Inference-Time RTC	新旧动作块断裂	动作前缀条件化与 guidance	推理开销、前缀可行性
Training-Time RTC	推理 guidance 开销	训练时学习前缀条件生成	需训练适配，延迟范围影响泛化
VLASH	本体状态滞后	状态前推 + 时间偏移微调	状态预测误差；图像仍可能滞后
FutureRTC	本体及视觉条件滞后	未来状态/视觉特征预测	预测误差；不可预见外部变化
RDP	块内部触觉反应慢	Slow–Fast 分层闭环	传感器、数据与控制接口要求
RTR	高频动作抖动	Reuse-then-Refine	与 RTC 不完全同一问题

4.6 实际系统怎样组合

对于带相机、手部本体反馈与触觉的机器人，推荐先按**系统层级**定位瓶颈：

1. 确定底层位置/速度/力控制频率、时间戳与安全限幅；验证执行器能够稳定追踪目标。
2. 在 VLA 或生成式动作策略上设置 H，K，Δt_c 并实测端到端及尾部延迟，而不是只报告 GPU 前向时间。
3. 发生块间跳变时考察 RTC；发生明显状态时间错位时考察 VLASH / FutureRTC；接触事件期间响应过慢时考察 RDP 一类高频反馈机制。
4. 分别评估动作连续性、碰撞/力峰值、延迟分布、干预后恢复率以及任务成功率。方法组合是否获益需要实证检验。

本讲结论：模仿学习不仅是 “学会预测动作”。完整链路需要正确的监督分布、可表达多峰动作的生成器、对任务有条件的 VLA，以及能在机器人实际时钟下安全、及时执行的闭环系统。

参考文献与延伸阅读

1. Ross et al. **DAgger**, 2011. https://arxiv.org/abs/1011.0686
2. Kelly et al. **HG-DAgger**, 2018. https://arxiv.org/abs/1810.02890
3. Mandlekar et al. **Learning to Imitate Human Demonstrations via Interactive Interventions (IWR)**. https://sites.google.com/stanford.edu/iwr
4. Zhao et al. **Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ACT)**, 2023. https://arxiv.org/abs/2304.13705
5. Chi et al. **Diffusion Policy**, 2023. https://arxiv.org/abs/2303.04137
6. Lipman et al. **Flow Matching**, 2023. https://arxiv.org/abs/2210.02747
7. Kim et al. **OpenVLA**, 2024. https://arxiv.org/abs/2406.09246
8. Black et al. **π_0**, 2024. https://www.physicalintelligence.company/blog/pi0
9. Perreault et al. **FAST**, 2025. https://www.roboticsproceedings.org/rss21/p012.pdf
10. **Real-Time Chunking (RTC)**. https://arxiv.org/search/?query=Real-Time+Chunking&searchtype=all
11. **VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference**, 2025. https://arxiv.org/abs/2512.01031
12. **FutureRTC**, 2026. https://arxiv.org/abs/2607.24008
13. Xue et al. **Reactive Diffusion Policy**, 2025. https://arxiv.org/abs/2503.02881
14. **RTR (Reuse-then-Refine)**, further reading. https://proceedings.mlr.press/v306/wang26lj.html

许可说明：引用论文及图像遵循各自许可。